DESIGN AND ANALYSIS OF ALGORITHMS

Emergency Ambulance Route Optimization System

A graph-based shortest-path prototype comparing Dijkstra's Algorithm and A* Search

OFFLINE SIMULATION

DIJKSTRA + A*

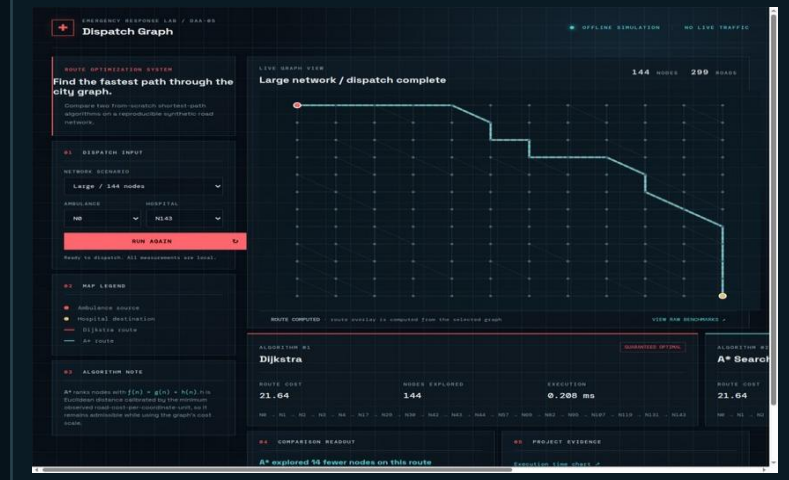
5th Semester • B.Tech Information Technology

Group: 6

Desai Mirant (12402080601083) • Patel Naitik (12402080601087)

Savani Nehal (12402080601092) • Dobariya Vivek (12402080601154)

Faculty: Yamini Dubey Ma'am



OPTIMAL ROUTE

Same graph • Same endpoints • Two algorithms

What problem are we solving?

Turn emergency routing into a measurable shortest-path problem.

PROBLEM

Find an optimal route from an ambulance source node to a hospital or emergency destination in a weighted road network.

Model the road network as a graph where intersections are nodes and roads are weighted edges.

INPUT

Synthetic weighted road graph
Ambulance / source node
Hospital / destination node
Positive edge costs

OUTPUT

Optimal route / node sequence
Total route cost
Nodes explored
Measured execution time
Side-by-side Dijkstra vs A* comparison

OBJECTIVE

Minimize the sum of edge costs and compare the search behavior of two correct shortest-path algorithms.

Why does this problem matter?

Emergency routing is a practical shortest-path application.

REAL-WORLD CONTEXT

Emergency vehicles may have multiple road choices.
A route planner can evaluate alternatives systematically.
Distance or estimated travel cost can become graph edge weights.

ROUTE OPTIMIZATION

The same graph abstraction applies to intersections, hospitals, emergency sites and roads.

DAA VALUE

Makes algorithmic trade-offs visible
Connects asymptotic analysis with measurements
Shows how search order changes explored nodes

KEY IDEA

Fewer explored nodes do not automatically mean lower wall-clock time.

PROJECT SCOPE

Offline simulation
Locally generated synthetic graph
No APIs or live traffic
Explainable algorithms

NOT A PRODUCTION DISPATCH SYSTEM

No GPS, verified road geometry, road closures or operational safety validation.

Project objectives

Each objective is tied directly to the DAA problem.

01 MODEL THE ROAD NETWORK

Represent intersections and roads as a weighted adjacency-list graph.

02 IMPLEMENT DIJKSTRA

Use a binary min-heap and predecessor-based path reconstruction.

03 IMPLEMENT A*

Use f(n)=g(n)+h(n) with a graph-calibrated Euclidean heuristic.

04 EXPOSE ROUTE METRICS

Show route, cost, explored nodes and execution time.

05 TEST CORRECTNESS

Cover normal, boundary and failure behavior with automated tests.

06 MEASURE PERFORMANCE

Compare both algorithms over multiple graph sizes and repeated trials.

Existing approach & need

The shortest-path problem provides the algorithmic foundation.

WEIGHTED SHORTEST PATH

Given G=(V,E), find a minimum-cost sequence from source s to destination t.

Dijkstra and A* are suitable because the generated graph uses positive / non-negative edge costs.

LIMITATION OF COST-ONLY SEARCH

Dijkstra is systematic and optimal, but it does not use geometric information to prefer nodes that appear closer to the goal.

A* ADDS DIRECTION

PROPOSED COMPARISON

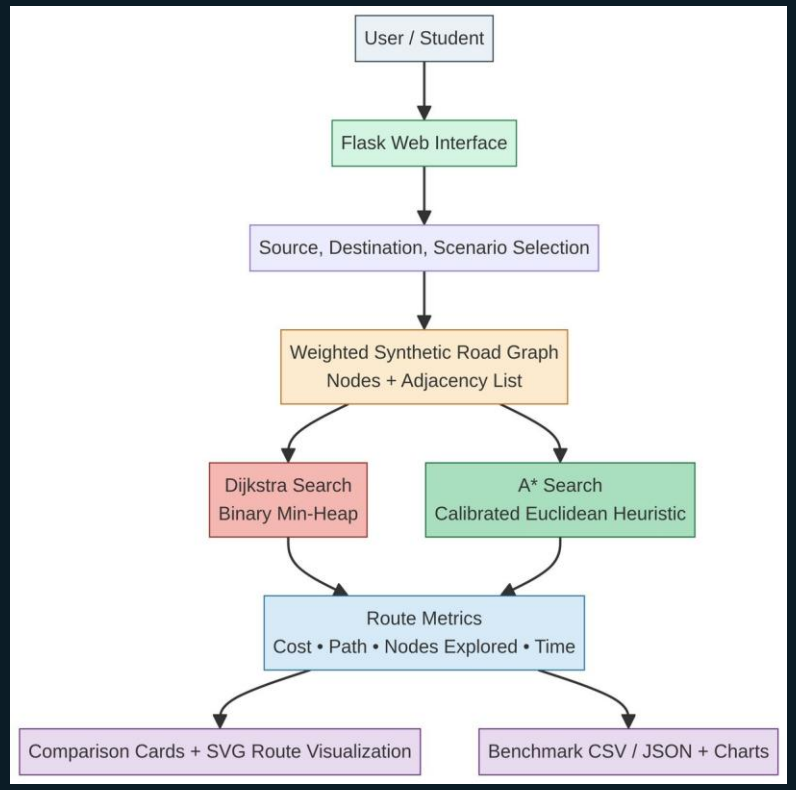
Dijkstra	Cost-so-far priority	No heuristic	Correctness baseline
A*	f(n)=g(n)+h(n)	Graph-calibrated Euclidean	Goal-directed search

Need for the proposed approach:

Keep both algorithms on the same graph and endpoints so the comparison is fair.

How the system works

Thin web layer; visible algorithmic core.



Architecture:user → Flask UI → graph → Dijkstra + A* → route metrics → visualization + benchmark evidence

Algorithms used

Two from-scratch shortest-path algorithms share the same graph.

01 Dijkstra

GUARANTEED OPTIMAL

Idea

Expand the unsettled node with the smallest known cost.
Relax outgoing edges and update distances.
Stop when the destination is removed from the heap.

Priority

g(n)

Core structure: adjacency list + binary min-heap

02 A* Search

GOAL - DIRECTED

Idea

Combine cost-so-far with an estimate to the goal.
Use the calibrated Euclidean heuristic.
Prioritize nodes with the smallest estimated total cost.

Priority

f(n)=g(n)+h(n)

Core structure: same graph + coordinate heuristic

Pseudocode highlights

The slides show the logic; the complete source remains in the project.

DIJKSTRA

```
distance[source]=0
heap ← (0,source)
visited ← ∅

while heap notem pty:
 extractm inim um -costnode
 mark visited
 ifcurrent= goal:break
 foreach neighbor:
    candidate =cost+ weight
    ifcandidate <distance[neighbor]:
       update distance & parent
       push candidate

return reconstructed path +cost
```

Complexity Worst case: O((V+E) log V)

A* SEARCH

```
g[source]=0
heap ← (h(source,goal),0,source)
visited ← ∅

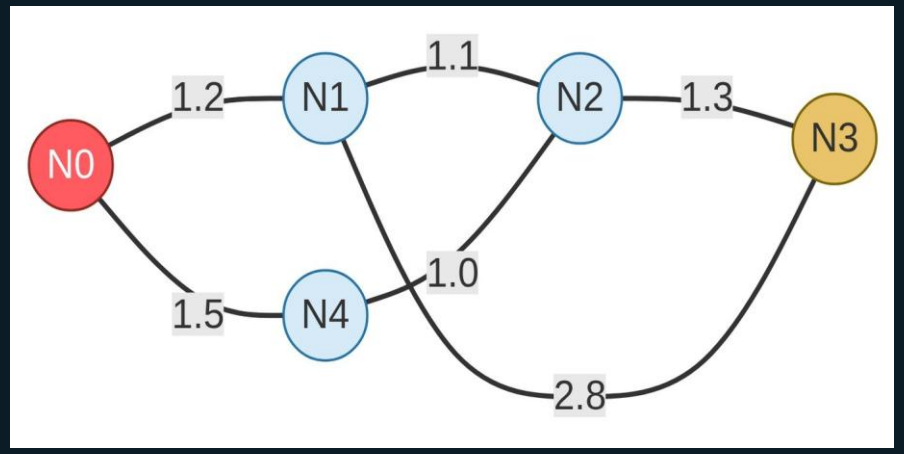
while heap notem pty:
 extractm inim um -fnode
 mark visited
 ifcurrent= goal:break
 foreach neighbor:
    candidate =g +weight
    ifcandidate <g[neighbor]:
       update g & parent
       f= candidate +h(neighbor,goal)
       push (f,candidate,neighbor)

return reconstructed path +cost
```

Heuristic h(a,b)=scale × EuclideanDistance(a,b)

Graph model & data structures

The graph is generated locally and stored as an adjacency list.



Illustrative model

Nodes = intersections
Edges = roads
Weights = route cost
N0 = source • N3 = destination

DATA STRUCTURES USED

Node record	id, x, y, label
Adjacency list	neighbor + weight pairs
Distance / g-cost	best known source cost
Predecessor map	route reconstruction
Binary min-heap	next lowest-priority node
Visited set	finalized node tracking

Implementation & working model

Python + Flask frontend around a visible algorithmic core.

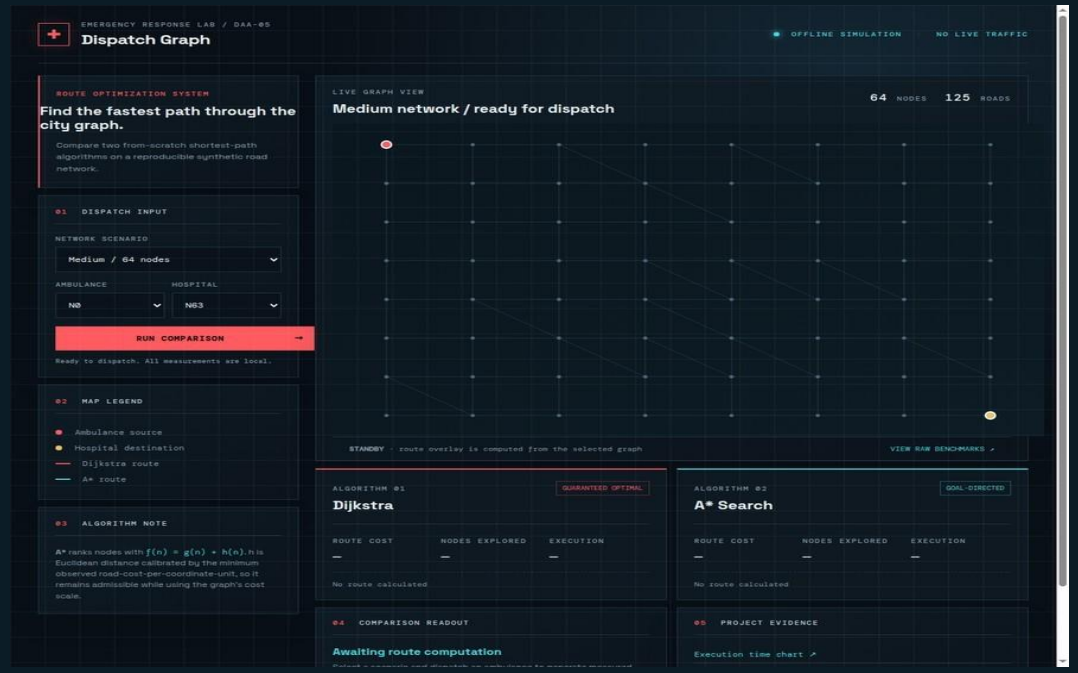
TECHNOLOGY STACK

Python	Application + algorithms
Flask	Local web server + JSON routes
HTML / CSS / JS	Dashboard + SVG visualization
pytest	Automated tests
matplotlib	Benchmark charts
heapq / time / csv / json	Priority queue, timing, evidence

KEY MODULES

app.py • graph/graph.py • graph/data.py
algorithms/dijkstra.py • algorithms/astar.py
benchmarking/benchmark.py • tests/test_algorithms.py

LIVE DEMONSTRATION



Select scenario → source → destination → RUN COMPARISON

Working project demonstration

Evidence from the actual local Flask application.

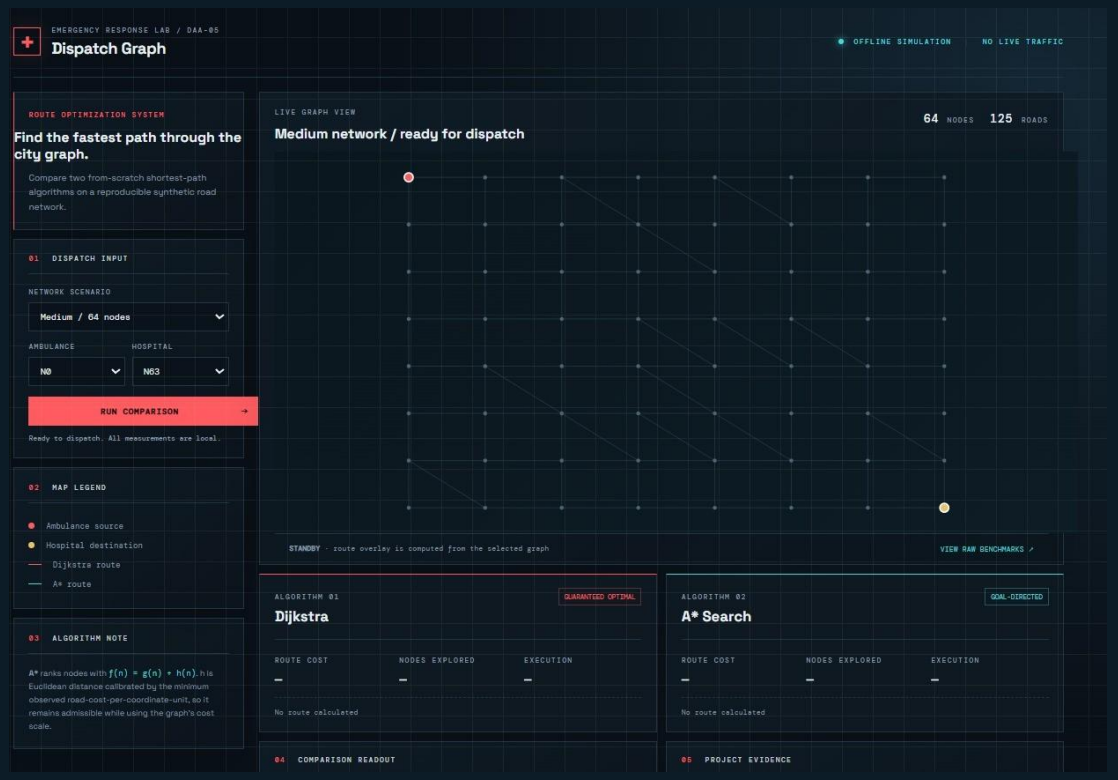


Figure A • Input: scenario and endpoints

Medium / 64 nodes, ambulance N0, hospital N63, ready to run.

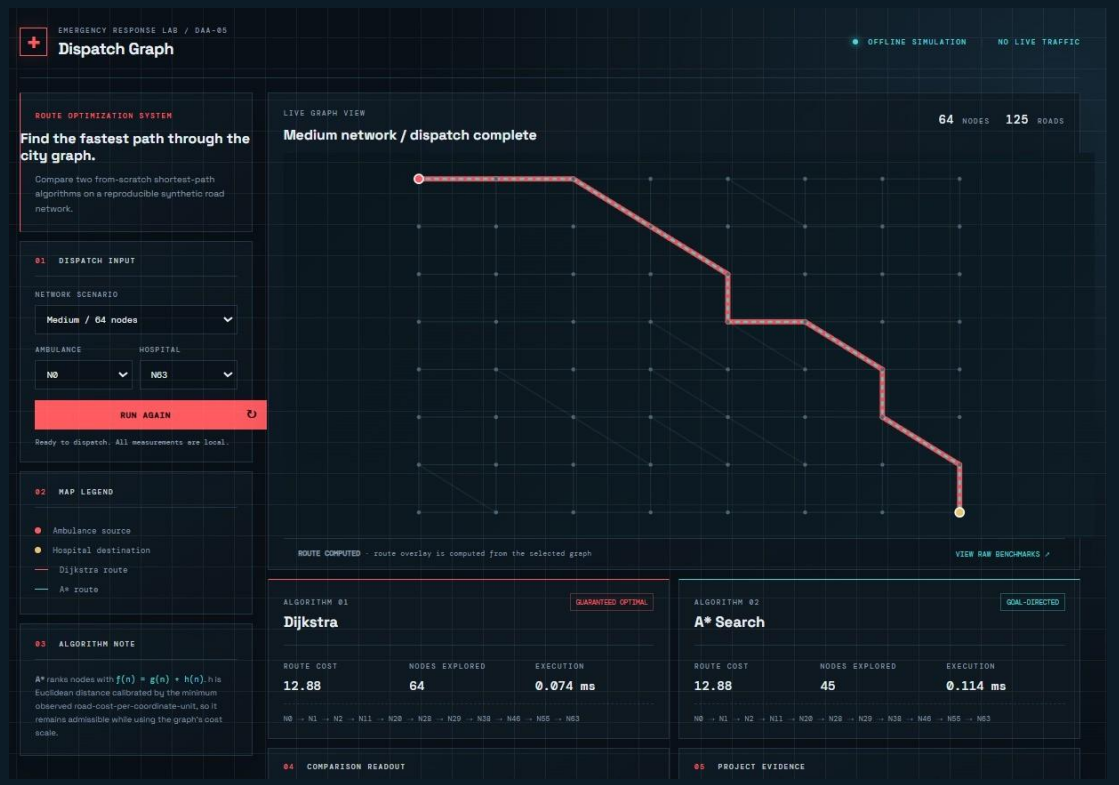


Figure B • Output: computed route and comparison

Both return cost 12.88; A* explores 45 nodes vs Dijkstra's 64.

Results & performance

56 measurements: 4 input sizes × 2 algorithms × 7 trials on deterministic synthetic grid graphs (averages shown).

Input (nodes)	Dijkstra (ms)	A* (ms)	Dijkstra nodes	A* nodes	Route cost (both)
10	0.0120	0.0111	7.29	4.00	3.0393
50	0.0440	0.0305	33.43	12.86	7.6555
100	0.1125	0.1270	100.00	64.14	16.2637
500	0.6367	0.6420	482.29	295.57	34.3338

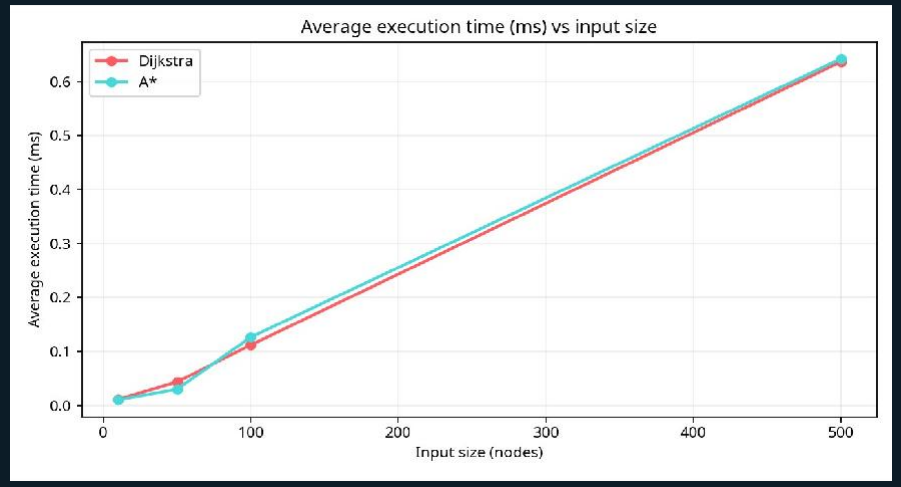


Figure 6 • Average execution time vs input size

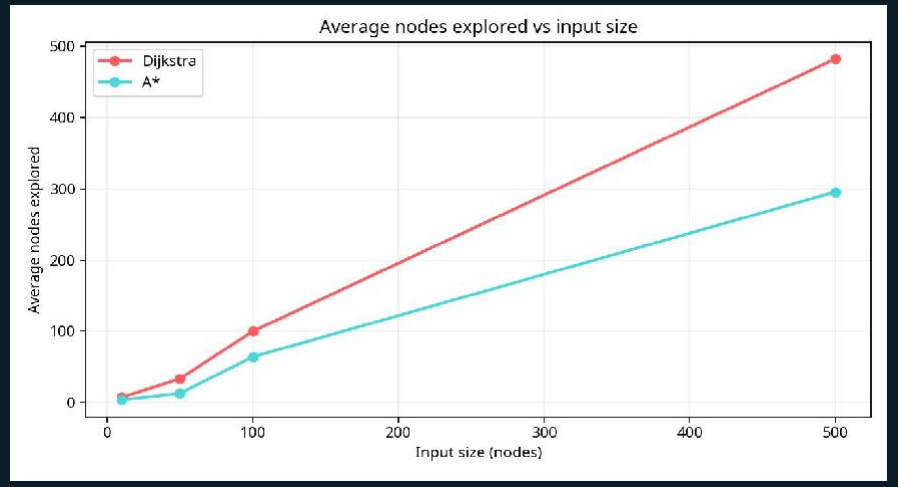


Figure 7 • Average nodes explored vs input size

Finding: A* explores fewer nodes at every size and returns the same optimal route cost.
Time is mixed: A* is faster at 10 and 50 nodes, slightly slower at 100 and 500 (heuristic and heap overhead in Python).

Complexity analysis

Derived from the actual adjacency-list + binary min-heap implementation.

Algorithm	Best	Average	Worst	Space
Dijkstra	O(1)*	Input-dependent	O((V+E) log V)	O(V+E) total
A*	O(1)*	Input / heuristic dependent	O((V+E) log V)	O(V+E) total

** Special case: source = destination → immediate termination.*

WHY THE SAME WORST-CASE BOUND?

Both use an adjacency list.
Both use a binary min-heap.
A* changes priority using h(n); its worst-case exploration can still be broad.

THEORETICAL vs PRACTICAL

Asymptotic bounds describe scaling, not exact milliseconds.
Node count reflects search behavior.
Runtime includes Python + heap + dictionary overhead.
Seven-trial averages reduce, but do not remove, timing noise.

V = vertices
E = edges

Advantages, limitations & future scope

What the prototype does well, where it stops, and where it can go next.

ADVANTAGES

- Both algorithms are written from scratch, so every step is explainable.
- Same graph and endpoints for both, giving a fair, reproducible comparison.
- A* explores fewer nodes (295.57 vs 482.29 at 500 nodes) at the same optimal cost.
- Dashboard shows both routes, costs and metrics side by side.
- 6 automated tests support correctness and heuristic admissibility.

LIMITATIONS

- Synthetic graph and simulated edge costs, not verified road geometry.
- No GPS, live traffic or road closures.
- Timings are machine-dependent and in Python, so A* is not always faster in wall-clock time.
- Local Flask prototype, not a dispatch-grade GIS system.

FUTURE SCOPE

- Integrate verified road data with time-dependent traffic.
- Dynamic rerouting and incident modeling.
- Multiple ambulances and multiple hospitals.
- Geographic map visualization.

Conclusion & references

Evidence-based DAA result, not a production dispatch claim.

CONCLUSION

The prototype successfully models emergency routing as a weighted shortest-path problem and compares two from-scratch algorithms on the same graph.

Dijkstra provides the cost-only baseline.
A* uses a calibrated Euclidean lower bound and explores fewer nodes in the measured datasets.
Both return the same route costs; runtime advantage depends on graph size and implementation overhead.

FINAL TAKEAWAY

The project demonstrates the complete DAA workflow: model → design → implement → test → measure → interpret.

REFERENCES

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Introduction to Algorithms (shortest-path chapters), MIT Press.
- E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," Numerische Mathematik, 1(1), 1959.
- P. E. Hart, N. J. Nilsson, B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," IEEE Trans. SSC, 4(2), 1968.
- Python documentation: heapq, time, csv, json (docs.python.org).
- Flask (flask.palletsprojects.com), pytest (docs.pytest.org), Matplotlib (matplotlib.org) documentation.
- Dataset: locally generated synthetic grid graphs (seeded, benchmark.py); project source and results/benchmark_summary.json.